

4.4 Examples

Here is an example demonstrating almost everything we've seen so far:

```
int globalvar = 1;
extern int anotherglobalvar;
static int privatevar;

f()
{
    int localvar;
    int localvar2 = 2;
    static int persistentvar;
}
```

Here we have six variables, three declared outside and three declared inside of the function `f()`.

`globalvar` is a global variable. The declaration we see is its defining instance (it happens also to include an initial value). `globalvar` can be used anywhere in this source file, and it could be used in other source files, too (as long as corresponding external declarations are issued in those other source files).

`anotherglobalvar` is a second global variable. It is *not* defined here; the defining instance for it (and its initialization) is somewhere else.

`privatevar` is a ``private" global variable. It can be used anywhere within this source file, but functions in other source files cannot access it, even if they try to issue external declarations for it. (If other source files try to declare a global variable called ```privatevar`", they'll get their own; they won't be sharing this one.) Since it has static duration and receives no explicit initialization, `privatevar` will be initialized to 0.

`localvar` is a local variable within the function `f()`. It can be accessed only within the function `f()`. (If any other part of the program declares a variable named ```localvar`", that variable will be distinct from the one we're looking at here.) `localvar` is conceptually ``created" each time `f()` is called, and disappears when `f()` returns. Any value which was stored in `localvar` last time `f()` was running will be lost and will not be available next time `f()` is called. Furthermore, since it has no explicit initializer, the value of `localvar` will in general be garbage each time `f()` is called.

`localvar2` is also local, and everything that we said about `localvar` applies to it, except that since its declaration includes an explicit initializer, it will be initialized to 2 each time `f()` is called.

Finally, `persistentvar` is again local to `f()`, but it *does* maintain its value between calls to `f()`. It has static duration but no explicit initializer, so its initial value will be 0.

The defining instances and external declarations we've been looking at so far have all been of simple variables. There are also defining instances and external declarations of functions, which we'll be looking at in the next chapter.

(Also, don't worry about static variables for now if they don't make sense to you; they're a relatively sophisticated concept, which you won't need to use at first.)

The term *declaration* is a general one which encompasses defining instances and external declarations; defining instances and external declarations are two different kinds of declarations. Furthermore, either kind of declaration suffices to inform the compiler of the name and type of a

particular variable (or function). If you have the defining instance of a global variable in a source file, the rest of that source file can use that variable without having to issue any external declarations. It's only in source files where the defining instance hasn't been seen that you need external declarations.

You will sometimes hear a defining instance referred to simply as a ``definition," and you will sometimes hear an external declaration referred to simply as a ``declaration." These usages are mildly ambiguous, in that you can't tell out of context whether a ``declaration" is a generic declaration (that might be a defining instance or an external declaration) or whether it's an external declaration that specifically is not a defining instance. (Similarly, there are other constructions that can be called ``definitions" in C, namely the definitions of preprocessor macros, structures, and typedefs, none of which we've met.) In these notes, we'll try to make things clear by using the unambiguous terms *defining instance* and *external declaration*. Elsewhere, you may have to look at the context to determine how the terms ``definition" and ``declaration" are being used.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)